Microsoft AutoGen: Advanced Course v2

Multi-Provider Multi-Agent AI Systems

Version: 2.0 (Multi-Provider Edition)
Author: Alpax Prime 🦙
Date: 2025

Course Overview

This advanced course covers Microsoft's AutoGen framework with hands-on experience across three different LLM providers: **Ollama** (local), **DeepSeek** (cloud), and **MiniMax** (cloud). Students learn to build production-ready multi-agent systems with provider-agnostic patterns.

What's New in v2: - Three-provider comparison (Ollama, DeepSeek, MiniMax) - Provider selection decision framework - Cost optimization strategies - Performance benchmarking insights - Production deployment patterns

Prerequisites: - Python 3.10+ - Basic understanding of LLMs and APIs - AutoGen 0.2+ installed

---

Module 1: AutoGen Fundamentals Review

1.1 Core Concepts

AutoGen enables **multi-agent applications** where AI agents communicate to accomplish tasks:

- **AssistantAgent**: AI-powered agent with LLM capabilities
- **UserProxyAgent**: Human interaction point
- **GroupChat**: Multiple agents in conversation
- **GroupChatManager**: Orchestrates group discussions

1.2 Agent Communication Patterns

```
┌──────────────┐      ┌──────────────┐
│  Assistant   │──────►│  UserProxy   │
│   Agent      │◄──────│   Agent      │
```

```
└──────────────┘      └──────────────┘

┌──────────────┐    ┌──────────────┐    ┌──────────────┐
│  Assistant   │────►│  Assistant   │────►│ UserProxy    │
│   Agent 1    │◄────│   Agent 2    │◄────│   Agent      │
└──────────────┘    └──────────────┘    └──────────────┘
```

Module 2: Provider Configuration Deep Dive

2.1 Ollama (Local)

Best For: Development, privacy-sensitive work, offline use

```python
from autogen import AssistantAgent, UserProxyAgent

config_list = [{
    "model": "qwen2.5-coder:7b",
    "base_url": "http://localhost:11434/v1",
    "api_key": "ollama",
    "api_type": "openai"
}]

assistant = AssistantAgent(
    name="assistant",
    llm_config={"config_list": config_list}
)

user_proxy = UserProxyAgent(
    name="user_proxy",
    human_input_mode="NEVER"
)
```

Setup:

```bash
# Install Ollama
brew install ollama

# Pull model
ollama pull qwen2.5-coder:7b

# Verify
ollama list
```

Pros: Zero cost, privacy, no rate limits
Cons: Requires hardware, limited models, slower

2.2 DeepSeek (Cloud)

Best For: Production, cost-sensitive projects, coding tasks

```python
import os
from dotenv import load_dotenv

load_dotenv()

config_list = [{
    "model": "deepseek-chat",
    "base_url": "https://api.deepseek.com/v1",
    "api_key": os.getenv("DEEPSEEK_API_KEY"),
}]

assistant = AssistantAgent(
    name="assistant",
    llm_config={"config_list": config_list}
)
```

Sign Up: https://platform.deepseek.com

Pros: Cheap, good coding, reliable
Cons: Internet required, costs accumulate

2.3 MiniMax (Cloud)

Best For: Long-context tasks, document processing

```python
import os
from dotenv import load_dotenv

load_dotenv()

config_list = [{
    "model": "MiniMax-Text-01",
    "base_url": "https://api.minimax.io/anthropic/v1",
    "api_key": os.getenv("MINIMAX_API_KEY"),
}]

assistant = AssistantAgent(
    name="assistant",
    llm_config={"config_list": config_list}
)
```

Sign Up: https://platform.minimax.io

Pros: 200K context, competitive pricing
Cons: Less known, documentation gaps

2.4 Provider Comparison Matrix

Feature	Ollama	DeepSeek	MiniMax
Type	Local	Cloud	Cloud

Feature	Ollama	DeepSeek	MiniMax
Cost	$0	|	
Context	4K-128K	64K	200K
Setup	Complex	Easy	Easy
Privacy	✅ Max	❌ None	❌ None
Speed	Variable	Fast	Fast
Offline	✅ Yes	❌ No	❌ No

Module 3: Lab Walkthroughs

Lab 1: Hello AutoGen (All Providers)

Objective: Verify installation and API connectivity

Steps: 1. Install dependencies 2. Configure provider 3. Create simple agent 4. Send test message

Ollama Code:

```
from autogen import AssistantAgent

config_list = [{"model": "qwen2.5-coder:7b",
                "base_url": "http://localhost:11434/v1",
                "api_key": "ollama"}]

agent = AssistantAgent("assistant", llm_config={"config_list":
        config_list})
response = agent.generate_reply(messages=[{"role": "user", "content":
        "Say hello"}])
print(response)
```

Verification: Each provider should return a greeting

Lab 2: Two-Agent Conversations

Objective: Set up conversation between agents

Architecture:

```
from autogen import AssistantAgent, UserProxyAgent

# Create assistant
assistant = AssistantAgent(
    name="writer",
    system_message="You are a creative writer.",
    llm_config={"config_list": config_list}
```

```
)

# Create user proxy
user_proxy = UserProxyAgent(
    name="user_proxy",
    human_input_mode="NEVER"
)

# Initiate conversation
user_proxy.initiate_chat(
    assistant,
    message="Write a short story about AI."
)
```

Provider Differences: - **Ollama**: Slower, but works offline - **DeepSeek**: Fast, good story quality - **MiniMax**: Excellent context handling

Lab 3: Tools & Functions

Objective: Register and use custom tools

```
from autogen import AssistantAgent, UserProxyAgent

def get_weather(location):
    """Get weather for a location"""
    return f"Weather in {location}: 72°F, sunny"

config_list = [{"model": "qwen2.5-coder:7b",
                "base_url": "http://localhost:11434/v1",
                "api_key": "ollama"}]

assistant = AssistantAgent(
    name="assistant",
    llm_config={"config_list": config_list}
)

# Register tool
assistant.register_for_execution()(get_weather)
assistant.register_for_llm()(get_weather)

# Use tool
user_proxy = UserProxyAgent(name="user", human_input_mode="NEVER")
user_proxy.initiate_chat(assistant, message="What's the weather in
        Tokyo?")
```

Key Points: - Functions must have docstrings (LLM uses them) - Register both for execution and LLM - Tool results passed back to LLM

Lab 4: GroupChat

Objective: Multi-agent group conversation

```python
from autogen import AssistantAgent, UserProxyAgent, GroupChat,
        GroupChatManager

# Create agents with different roles
writer = AssistantAgent(name="writer", system_message="Creative
        writer")
editor = AssistantAgent(name="editor", system_message="Expert editor")
critic = AssistantAgent(name="critic", system_message="Constructive
        critic")

# Create group chat
groupchat = GroupChat(
    agents=[writer, editor, critic],
    messages=[],
    max_round=10
)

# Create manager
manager = GroupChatManager(groupchat=groupchat, llm_config=
        {"config_list": config_list})

# Start group conversation
user_proxy = UserProxyAgent(name="user", human_input_mode="NEVER")
user_proxy.initiate_chat(manager, message="Create a short poem about
        technology")
```

Speaker Selection Strategies: - `auto`: AutoGen decides - `round_robin`: Equal turns - `fixed`: Predefined order

Lab 5: Human-in-the-Loop

Objective: Add human approval to workflows

```python
from autogen import AssistantAgent, UserProxyAgent

# Always ask humans
human_proxy = UserProxyAgent(
    name="human",
    human_input_mode="ALWAYS"  # Options: ALWAYS, NEVER, TERMINATE
)

# Or ask only at termination
approval_proxy = UserProxyAgent(
    name="approver",
    human_input_mode="TERMINATE"
)

assistant = AssistantAgent(
    name="assistant",
    llm_config={"config_list": config_list}
)
```

```
# Workflow with human approval
human_proxy.initiate_chat(
    assistant,
    message="Write code to delete all files"
)
# Human will be prompted before execution
```

Modes: - `ALWAYS`: Every message requires human input - `NEVER`: Fully automated - `TERMINATE`: Ask at conversation end

Lab 6: Real-World Applications

Objective: Build production-ready workflows

```
from autogen import AssistantAgent, UserProxyAgent
from autogen.agentchat.conversable_agent import ConversableAgent

# Research agent workflow
researcher = AssistantAgent(
    name="researcher",
    system_message="Research assistant that finds information",
    llm_config={"config_list": config_list}
)

summarizer = AssistantAgent(
    name="summarizer",
    system_message="Summarizes findings concisely",
    llm_config={"config_list": config_list}
)

# Sequential workflow
user_proxy = UserProxyAgent(name="user", human_input_mode="NEVER")

# Step 1: Research
user_proxy.initiate_chat(researcher, message="Research quantum
        computing")

# Step 2: Summarize (pass context)
user_proxy.initiate_chat(summarizer, message="Summarize the research
        findings")
```

Lab 7: Production Patterns

Objective: Scaling and monitoring

```
import logging
from autogen import AssistantAgent, UserProxyAgent

# Enable logging
logging.basicConfig(level=logging.INFO)
```

```python
# Error handling
from autogen import ConversableAgent

class ErrorHandlingAgent(ConversableAgent):
    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        self.max_retries = 3

    def generate_reply(self, messages):
        for attempt in range(self.max_retries):
            try:
                return super().generate_reply(messages)
            except Exception as e:
                logging.error(f"Attempt {attempt+1} failed: {e}")
                if attempt == self.max_retries - 1:
                    return "I encountered an error. Please try again."

# Retry configuration
assistant = AssistantAgent(
    name="robust_assistant",
    llm_config={
        "config_list": config_list,
        "timeout": 120,
        "cache_seed": None  # Disable caching for production
    }
)
```

Module 4: Provider Selection Framework

4.1 Decision Tree

```
Start
  │
  ▼
Need offline? ──YES──▶ Use Ollama
  │
  NO
  │
  ▼
Need long context? ──YES──▶ Use MiniMax (200K)
  │
  NO
  │
  ▼
Cost-sensitive? ──YES──▶ Use DeepSeek
  │
  NO
  │
```

```
  ▼
Use OpenAI/Anthropic
```

4.2 Use Case Mapping

Use Case	Recommended Provider
Development/Testing	Ollama
Production (low cost)	DeepSeek
Long documents	MiniMax
Coding tasks	DeepSeek/Ollama
Enterprise	MiniMax
Research	MiniMax

Module 5: Performance & Cost Optimization

5.1 Cost Comparison

Provider	Input/1M	Output/1M	Context
DeepSeek	$0.14	$0.28	64K
MiniMax	$0.10	$0.30	200K
GPT-4	$15.00	$60.00	128K
Claude	$15.00	$75.00	200K

5.2 Optimization Strategies

1. **Caching**: Enable for repeated queries
2. **Batching**: Group requests when possible
3. **Truncation**: Trim long contexts
4. **Fallback**: Chain providers for reliability

```python
# Fallback provider chain
def get_config_list(preferred="deepseek"):
    if preferred == "deepseek":
        return deepseek_config
    elif preferred == "minimax":
        return minimax_config
    else:
        return ollama_config
```

Module 6: Production Best Practices

6.1 Error Handling

```
try:
    response = agent.generate_reply(messages)
except Exception as e:
    logger.error(f"Agent error: {e}")
    # Implement retry or fallback
```

6.2 Logging

```
import logging

logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s - %(name)s - %(levelname)s - %(message)s'
)

# Agent logs
logger = logging.getLogger("autogen")
logger.setLevel(logging.DEBUG)
```

6.3 Monitoring

```
# Track metrics
from collections import defaultdict

class AgentMonitor:
    def __init__(self):
        self.request_count = 0
        self.error_count = 0
        self.total_tokens = 0

    def track(self, success, tokens):
        self.request_count += 1
        if not success:
            self.error_count += 1
        self.total_tokens += tokens
```

Module 7: Capstone Project

Build a Multi-Provider Research Assistant

Requirements: 1. Uses Ollama for quick local queries 2. Falls back to DeepSeek for production 3. Can use MiniMax for long-document analysis 4. Implements human-in-the-loop for sensitive tasks 5. Logs all interactions for debugging

Architecture:

```
┌──────────────┐
│  User Query  │
└──────────────┘
       │
       ▼
┌──────────────┐      ┌──────────────┐
│  Classifier  │──────▶│ Ollama (Fast)│
└──────────────┘      └──────────────┘
                             │
                        Response?
                             │
                      ┌──────┴──────┐
                      ▼             ▼
                ┌──────────┐   ┌──────────┐
                │ DeepSeek │   │ MiniMax  │
                │(Produce) │   │ (Long)   │
                └──────────┘   └──────────┘
                      │             │
                      └──────┬──────┘
                             │
                             ▼
                      ┌──────────────┐
                      │    Human     │
                      │  (Optional)  │
                      └──────────────┘
```

Conclusion

This course provided hands-on experience with three AutoGen providers:

- **Ollama**: Perfect for development and privacy
- **DeepSeek**: Best cost-to-performance ratio
- **MiniMax**: Ideal for long-context tasks

Key Takeaways: 1. Provider choice depends on use case 2. Design for provider independence 3. Implement fallbacks for reliability 4. Monitor costs in production 5. Use appropriate context windows

---

Appendix: Quick Reference

Provider Setup Checklist

- [ ] Ollama: `brew install && ollama pull qwen2.5-coder:7b`
- [ ] DeepSeek: Get API key from platform.deepseek.com
- [ ] MiniMax: Get API key from platform.minimax.io

Environment Variables

```
DEEPSEEK_API_KEY=sk-...
MINIMAX_API_KEY=sk-...
OPENAI_API_KEY=sk-...
```

Common Issues

Issue	Solution
Ollama not responding	Check port 11434, restart service
Rate limit errors	Implement retry with backoff
Context overflow	Truncate or summarize
Timeout errors	Increase timeout in config

Course Version 2.0 - Multi-Provider Edition
Built with Microsoft AutoGen 🦙